Polytech Clermont

Mathematical Engineering and Data Science, 4rd year

Tetris Game Design and Development

By : Etienne ESCOFIER, Andrea GELIE, Adrien LABAYLE, Thomas MAS, Camille RIVIERE, Aurélien SAIS

2025 /2026

This report presents the design and development of a Tetris video game made for a software engineering and object-oriented programming project. The main objective was to implement a functional version of the game using the C++ programming language and the Qt graphical library, while applying UML modeling principles.

The project followed a structured methodology, including task planning using a Gantt chart, system modeling through various UML diagrams, and finally the implementation of the game. Particular attention was paid to object-oriented design, especially through the definition of the main classes and the implementation of essential features such as collision management, full line removal, and score calculation.

The final result is a Tetris game featuring a graphical user interface, multiple difficulty levels, a progressive scoring system, as well as a save and resume functionality.

Keywords: Tetris; Object-oriented programming; UML; C++; Qt

TABLE OF CONTENTS

1 INTRODUCTION AND PROJECT OVERVIEW

As part of our Software Engineering and Object-Oriented Programming course, we were asked to develop a computer implementation of the Tetris video game.

The project required us to apply the concepts studied throughout the course, with a strong emphasis on object-oriented programming principles.

We also used UML modeling techniques to design the application and developed the game using the C++ programming language combined with the Qt graphical framework.

Several complementary documents were produced in order to support the development and understanding of the application, both for users and future developers.

2 GAME PRESENTATION

Tetris[1] is a puzzle video game originally created by Alexey Pajitnov in 1984.

The objective of the game is to complete horizontal lines using geometric blocks called tetrominoes

Figure 1: *Example of an endgame on the free clone of the game JsTetris[1]*

Players can move, rotate, and accelerate the falling pieces in order to maximize the score and avoid filling the entire grid.

The scoring system rewards players according to the number of completed lines, while increasing difficulty levels progressively accelerate gameplay.

3 PROJECT PLANNING

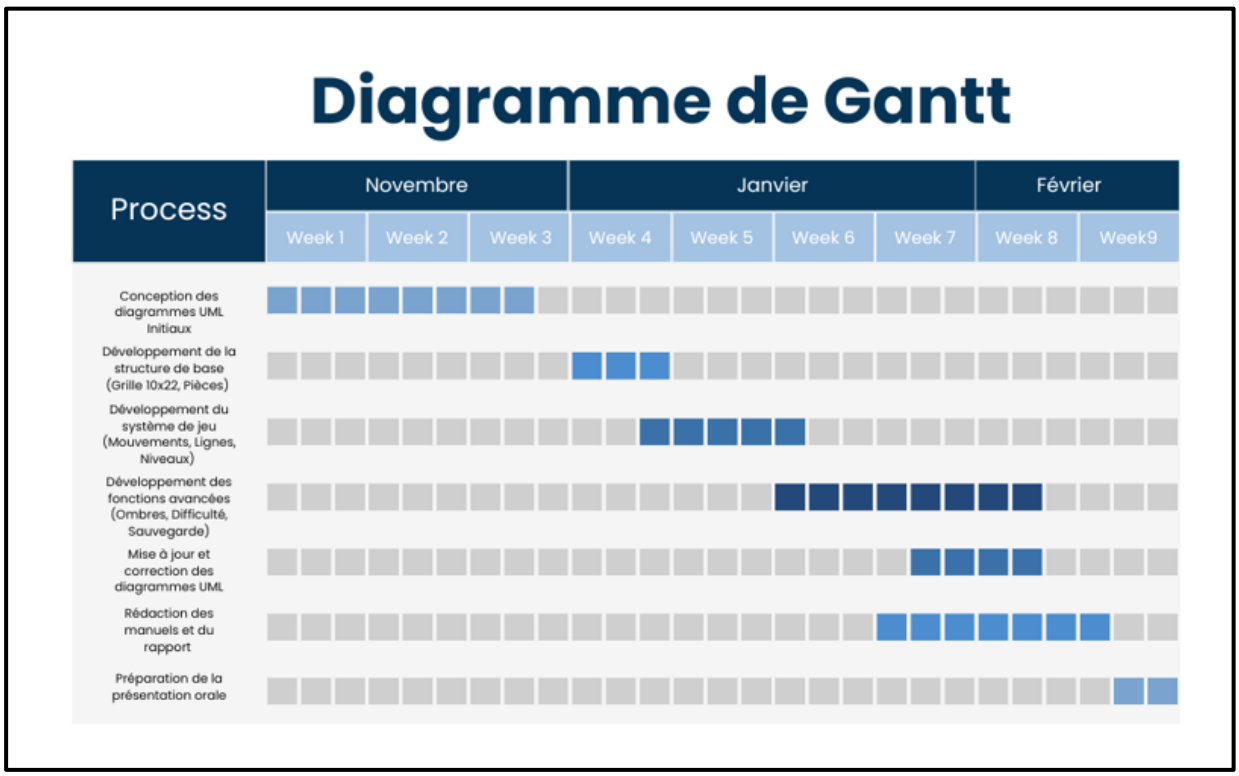

Figure 2 : *Gantt chart*

To organize the different stages of the project efficiently, we created a Gantt chart covering the UML design, implementation, and documentation phases.

The development process was divided into several stages: the Creation of the base game structure, the Implementation of gameplay mechanics and the Development of advanced features.

4 PRESENTATION OF THE TOOLS USED

Several software tools were used during the project:

- **Visual Paradigm[2]** and **Lucidchart [3]** for UML diagram creation
- **Qt Creator[4]** for graphical interface development in C++
- **Convertio[5]** for image conversion and adaptation
- **noTube[6]** for audio integration

5 UML DIAGRAMS AND KEY CODE COMPONENTS

5.1 UML DIAGRAMS

The UML diagrams played a major role in the development of the project because they allowed us to structure the application before starting the implementation phase. They helped us define the main functionalities of the game, organize the architecture of the software, and better understand the interactions between the player, the interface, and the game logic.

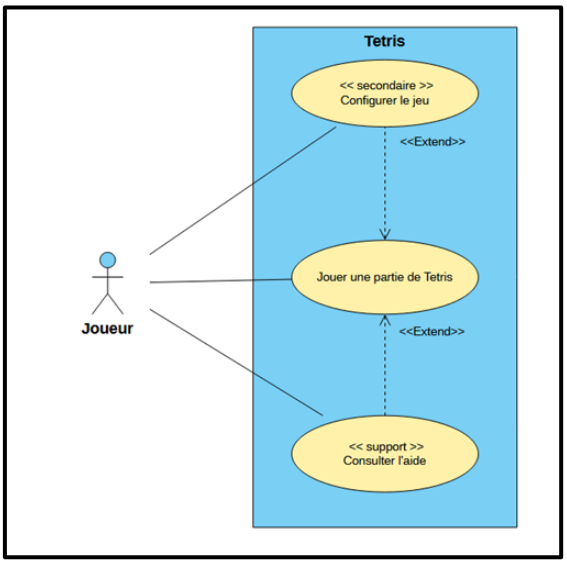

Figure 3 : *Use case diagram*

The use case diagram represents the main interactions between the player and the Tetris game. Its purpose is to provide a global overview of the functionalities available to the user.

The main use case is “Play a Tetris Game”, which corresponds to the central objective of the software. In addition to this primary functionality, the player can also configure the game difficulty and consult the help menu.

This diagram served as the starting point of the project design process. It helped us identify the essential features that needed to be implemented during the first stages of development. As the project progressed, additional functionalities were added, such as the ability to save and resume a game, even though they were not included in the initial version of the diagram.

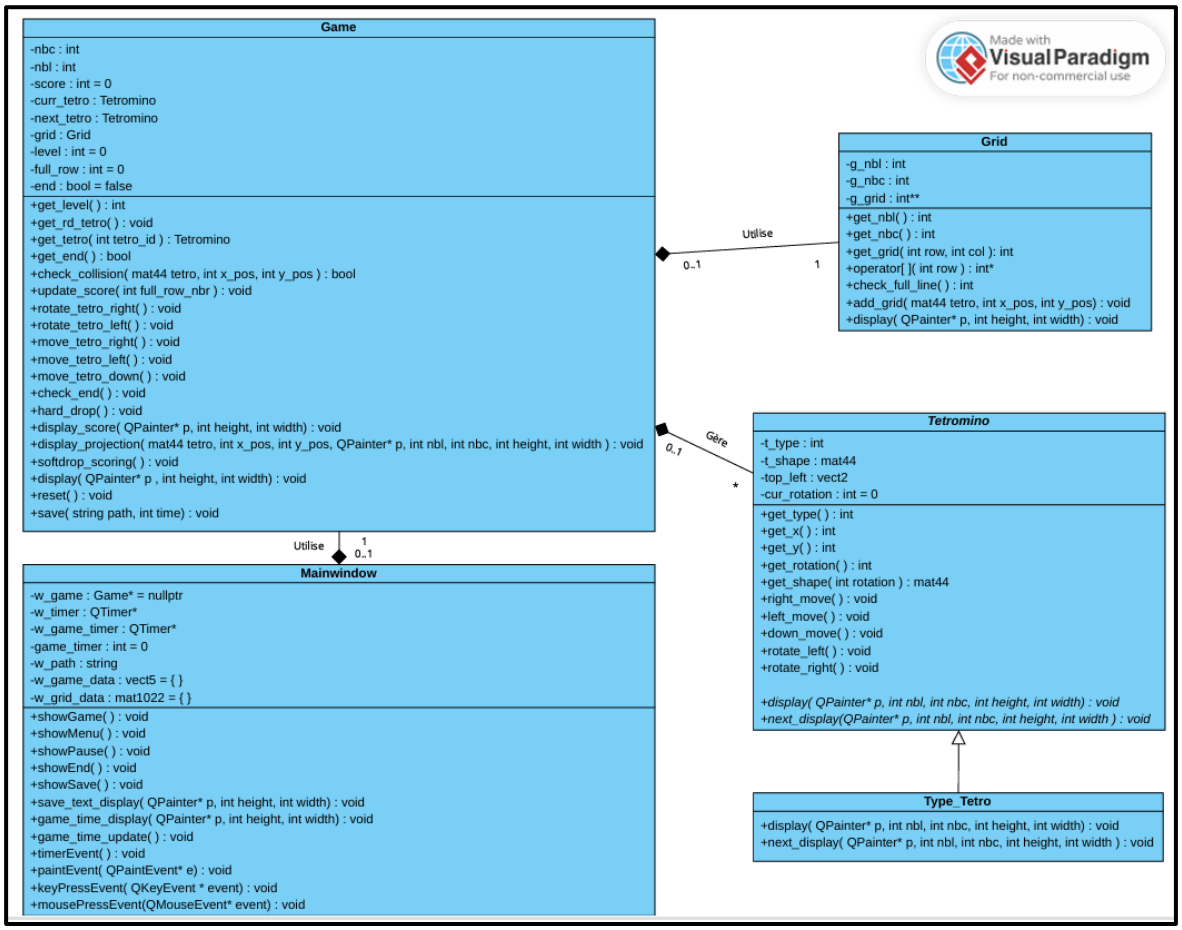

Figure 4 : *Class diagram*

The class diagram models the internal structure of the application by describing the main classes, their attributes, their methods, and the relationships between them.

The project is mainly composed of the following classes: Game, Grid, Tetromino, Type_Tetro and MainWindow.

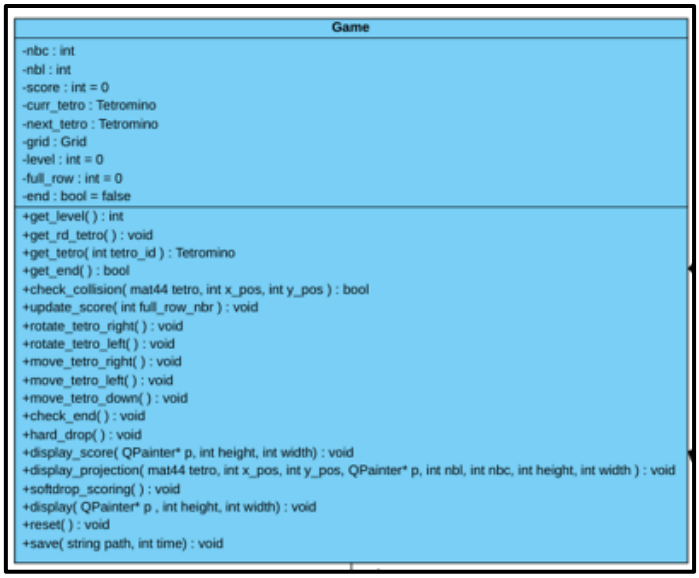

Figure 5 : *Game class*

The Game class is the core class of the application. It manages the overall game logic, including the current tetromino, the next tetromino, the score, the level, the timers, and the progression of the game.

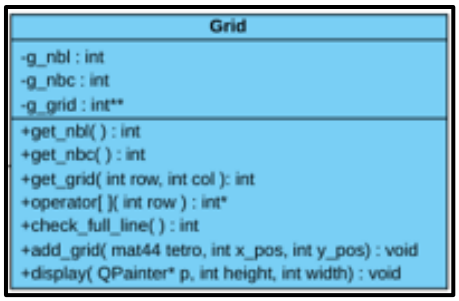

Figure 6 : *Grid class*

The Grid class manages the playing field. It stores the state of the grid and provides methods for checking collisions, detecting completed lines, updating the grid, and calculating the score.

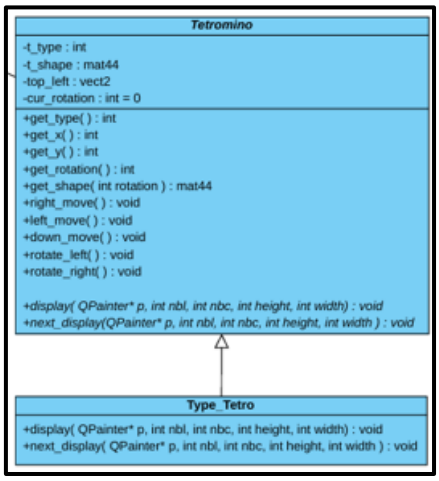

Figure 7 : *Tretromino class and heritage*

The Tetromino class is an abstract class that defines the common behavior shared by all tetrominoes. It contains the methods related to the management, movement, and display of the falling blocks.

The Type_Tetro subclasses represent the seven different tetromino shapes available in the game. Each subclass is associated with a specific tetromino and handles its display and rotation behavior.

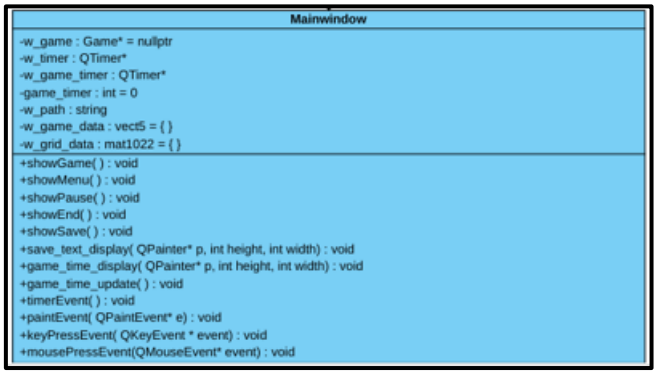

Figure 8 : *Mainwindow class*

Finally, the MainWindow class manages the graphical user interface using the Qt framework. It handles the display of menus, player interactions, keyboard inputs, timers, and communication between the interface and the game logic.

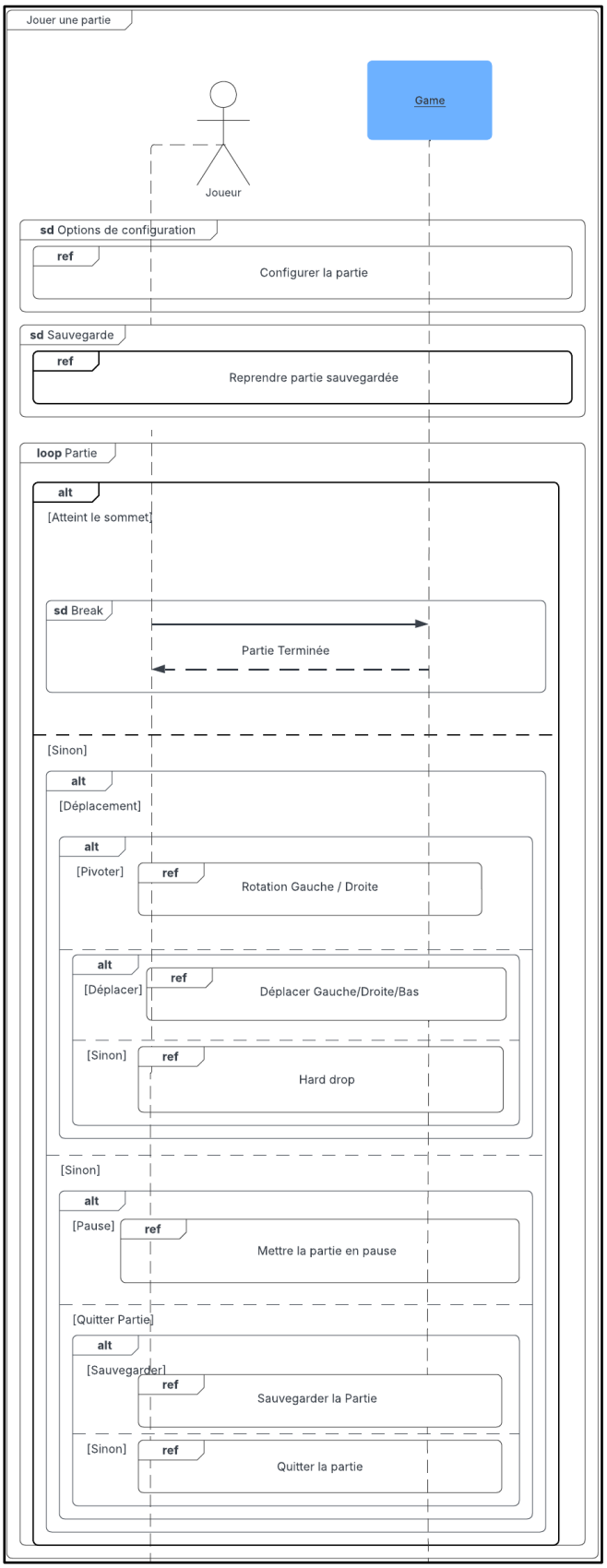

Figure 9 : *Sequence diagram of the entire game*

The sequence diagrams describe the dynamic interactions between the player and the different components of the application during gameplay.

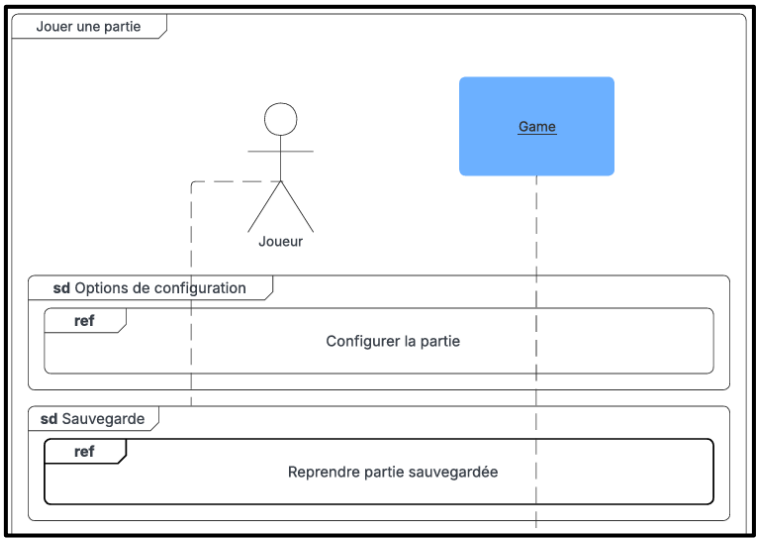

Figure 10 : *Part of the sequence diagram that does not concern the part*

The general sequence diagram illustrates the overall flow of the game, from the moment the player launches the application to the different interactions possible during a game. Before starting a new game, the player can configure the game settings, consult the help menu, or continue a previously saved game.

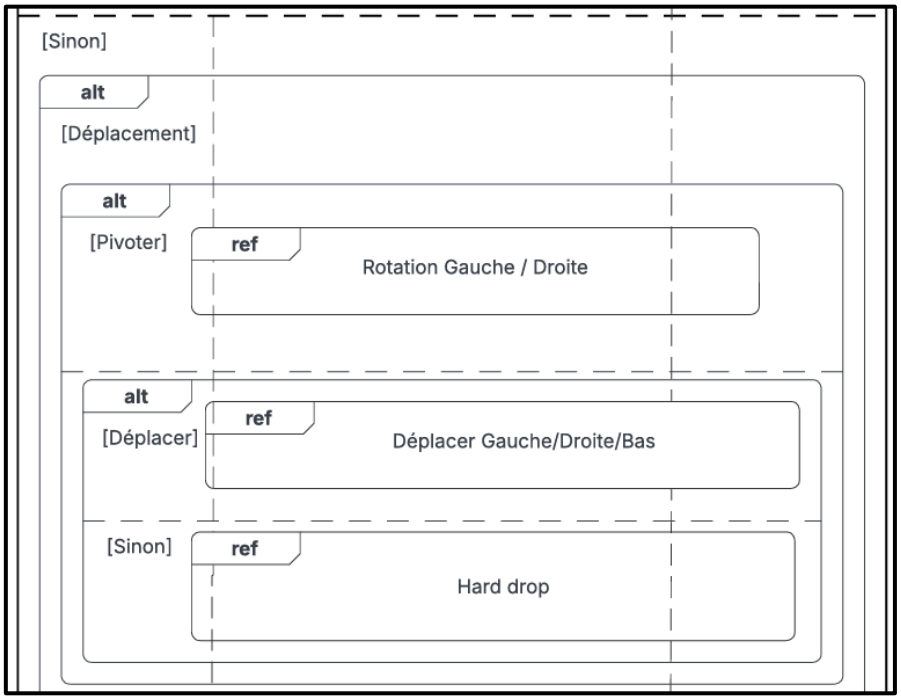

Figure 12 : *Various movements within the loop section*

During the gameplay loop, the player can perform several actions such as moving the current tetromino, rotating it, accelerating its fall, pausing the game, saving the game, or quitting the game.

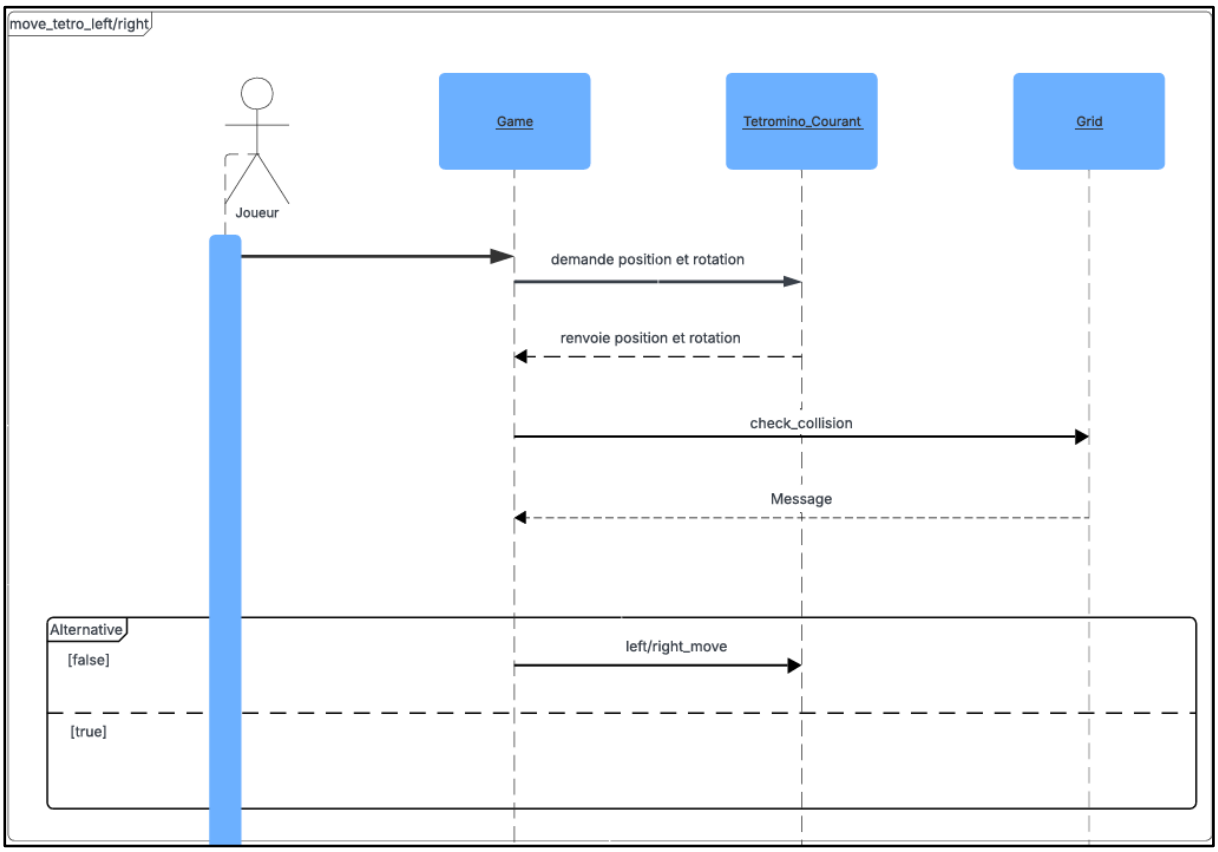

Figure 14 : *Left/right movement sequence diagram*

When the player moves a tetromino to the left or right, the game first retrieves the current position and rotation of the piece. The system then checks whether the requested movement would create a collision with either the boundaries of the grid or already placed tetrominoes. If a collision is detected, the movement is rejected. Otherwise, the tetromino is moved successfully.

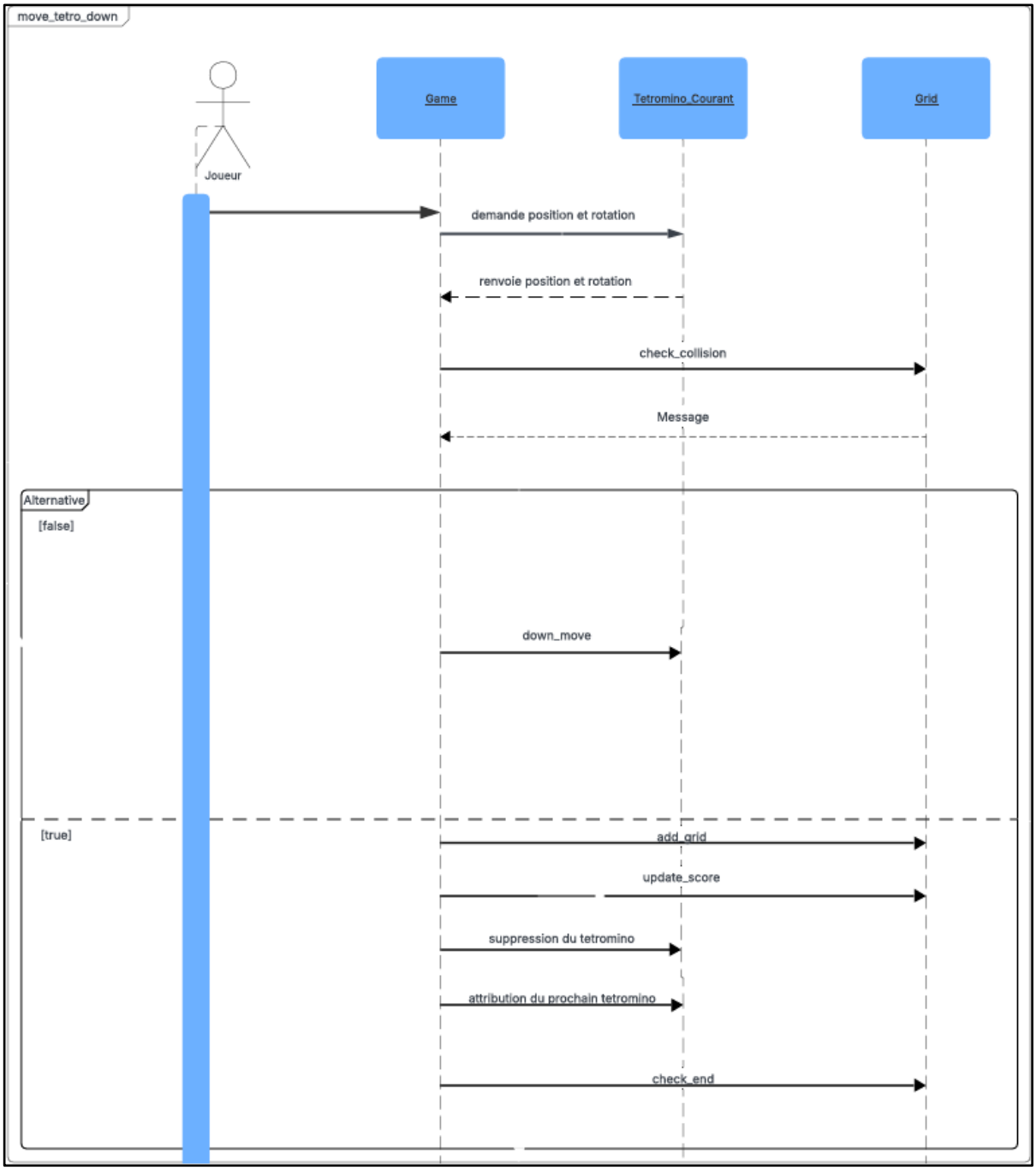

Figure 15 : *Downward movement sequence diagram*

The downward movement sequence works similarly at first. However, when a collision is detected below the tetromino, the piece is automatically fixed into the grid. The game then checks whether one or more lines are complete, updates the score accordingly, generates a new tetromino, and verifies whether the player has lost the game.

5.2 KEY CODE COMPONENTS

Several methods implemented in the code are essential to the correct functioning of the game.

The check_collision method is one of the most important methods in the project. It belongs to the Game class and is responsible for detecting collisions between tetrominoes and the playing field. This method receives as parameters a 4×4 matrix representing the current tetromino and the future coordinates of its upper-left corner.

The method verifies several conditions: whether the tetromino remains within the boundaries of the grid, whether it collides with already placed blocks, and whether it reaches the bottom of the playing field.

If one of these conditions is violated, the method returns true, meaning that a collision has been detected. Otherwise, it returns false, meaning that the movement is valid.

This method is called every time the player moves, rotates, or drops a tetromino. It therefore plays a fundamental role in ensuring that the gameplay remains coherent and functional.

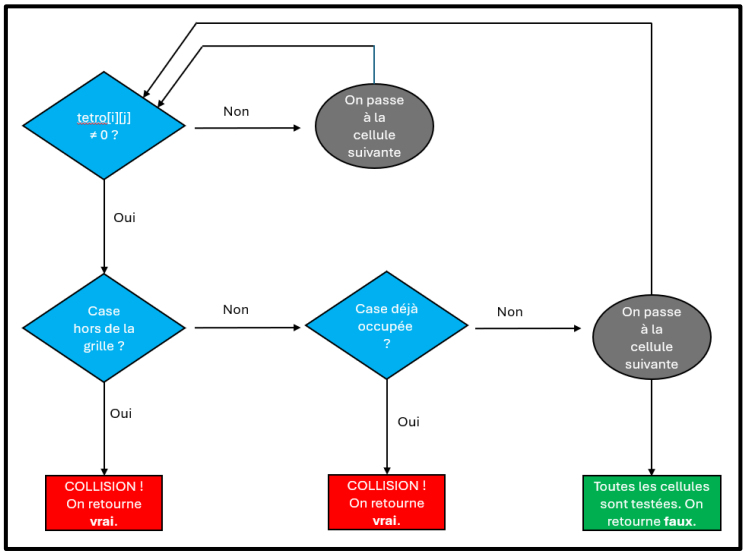

Figure 16 : *Diagram of how the check_collision method works*

The check_full_line method belongs to the Grid class and is responsible for detecting and removing completed lines. The method scans the grid from bottom to top. For each row, it checks whether all cells are occupied. If no empty cell is found, the row is considered complete. When a complete line is detected, the method removes it and shifts all rows located above it downward. This process is repeated for every complete line detected in the grid.

The method also returns the number of removed lines, which is then used to update the player's score and level. This functionality is essential because line clearing is one of the core mechanics of the Tetris game.

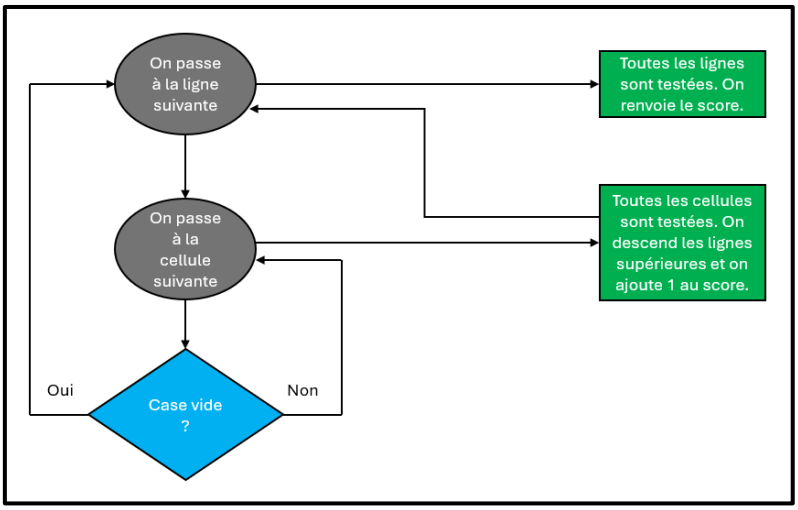

Figure 17 : *Diagram of how the check_full_line method works*

The display_projection method improves the gameplay experience by displaying a projection of the current tetromino. This projection represents the position where the tetromino would land if the player used the hard drop feature. The projection is displayed in grey on the grid. To calculate this position, the method creates a copy of the current tetromino and simulates its downward movement until a collision is detected. This feature allows players to better anticipate the final position of the tetromino and improves the overall playability of the game, especially for beginner players.

The save method allows the player to save the current state of the game. The game data is stored inside a text file containing all the information necessary to restore the game later. This includes the state of the grid, the current tetromino, the next tetromino, the score, the level, and other gameplay information. The save file can then be read by the continuation

function in order to restore the game exactly as it was before being closed. This feature allows players to interrupt a game and resume it later. However, only one save slot is available in the current version of the application.

6 PRESENTATION OF THE FINAL GAME

The final version of the game includes:

- A graphical user interface:

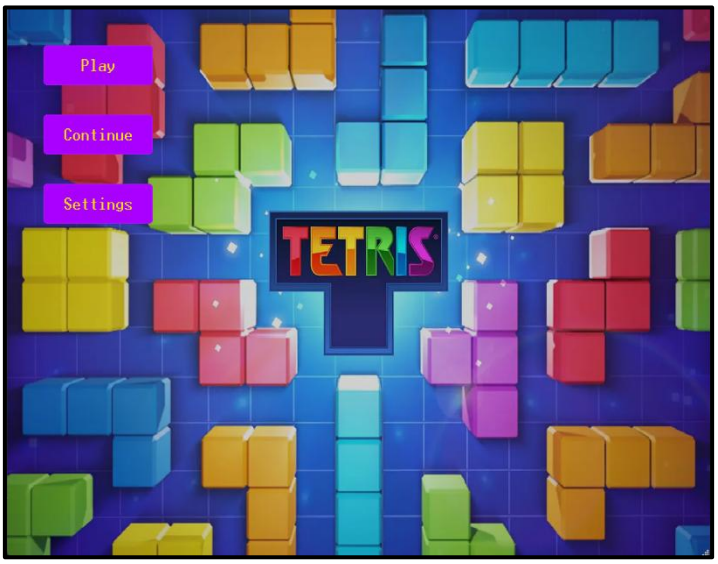

Figure 18 : *Main game menu*

- Several difficulty levels : Players can choose between Easy, Medium, and Hard difficulty modes before starting a game.

Figure 19 : *Difficulty selection*

- A score and level system :

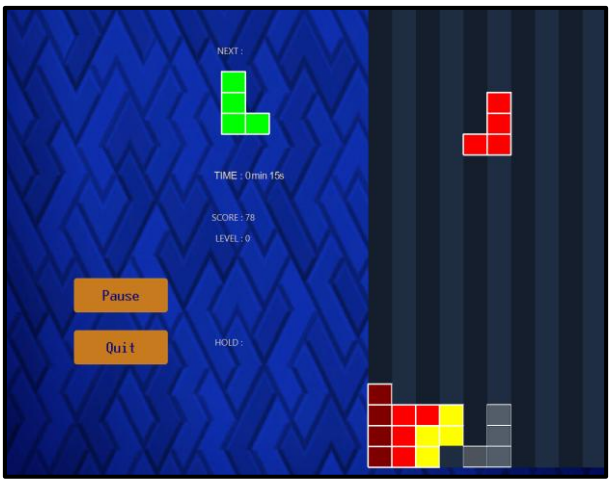

Figure 20 : *Configuration during a Tetris game*

- Pause and resume menus :

Figure 21 : *Pause during gameplay*

- Save and continue functionality ***:***

Figure 22 : *Display during a defeat*

7 PERSONAL REFLECTION

This project allowed us to strengthen both our technical and organizational skills.

We improved our proficiency in C++, Qt, UML modeling, and object-oriented programming concepts.

Several technical challenges were encountered during development, particularly regarding collision management, line clearing, and score updates.

Working as a team also helped us develop communication, organization, and project management skills.

CONCLUSION

The final version of the project successfully implements the core mechanics of Tetris, including piece movement, rotation, line clearing, score management, and multiple difficulty levels. Additional features such as game saving and tetromino projection further improved the user experience.

Future improvements could include a high-score tracking system and customizable visual themes.

BIBLIOGRAPHY

[1] « *Tetris* », *Wikipédia*. 7 novembre 2025. Consulté le: 30 janvier 2026. [En ligne]. Disponible sur: https://fr.wikipedia.org/w/index.php?title=Tetris&oldid=230437009

[2] « Visual Paradigm - AI-Powered Visual Modeling ». Consulté le: 30 janvier 2026. [En ligne]. Disponible sur: https://www.visual-paradigm.com/

[3] « Lucidchart | Diagrammes alimentés par l'intelligence », Lucidchart. Consulté le: 30 janvier 2026. [En ligne]. Disponible sur: https://www.lucidchart.com/pages/fr

[4] « Download Source Package Offline Installers | Qt ». Consulté le: 30 janvier 2026. [En ligne]. Disponible sur: https://www.qt.io/development/offline-installers

[5] « Convertir JPEG en JPG (En ligne et Gratuit) — Convertio ». Consulté le: 6 février 2026. [En ligne]. Disponible sur: https://convertio.co/fr/jpeg-jpg/

[6] « Convertisseur : YouTube MP3, Tiktok MP4 et plus encore! - noTube ». Consulté le: 6 février 2026. [En ligne]. Disponible sur: https://notube.lol/fr/youtube-app-311